EMIOT

Architecture

User input → LLM extracts JSON → Python performs database extraction and computation → C++ performs thermodynamic and physical simulation

The LLM performs minimal work, leveraging its understanding capabilities to handle imperfections in user input.

Python handles database extraction and computation: For small matrices, large models might produce correct results, but for large matrices,

large models cannot guarantee stable and correct output of all numbers without omitting a single character; their nature is probabilistic chaining. For complex topological results, such as real battery packs—which may not be perfect solid rectangles—large models are highly likely to misalign.

Additionally, there are token costs and latency issues, so using Python is a better choice.

Brainstorming

A: The Expert

Features: The user provides all key parameters.

Example: “I need a 4s3p battery pack, 4 groups with 3 cells each, S-type cooling, 1 channel, coolant length 9.”

JSON output:

```
{
  "intent": {
    "type": "custom",
    "search_keyword": "none"
  },
  "simulation_parameters": {
```

```
    "total_cells": null,
    "num_groups": 4,
    "cells_per_group": 3,
    "cooling_type": 0,
    "coolant_channels": 1,
    "coolant_size": [9]
  },
  "layout_features": {
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [
      "total_cells"
    ],
    "assumptions_made": "none"
  }
}
```

B: Template-based

Features: The user provides only a specific name.

User input: “Build a battery pack similar to the Tesla Model S.”

JSON output:

```
{
  "intent": {
    "type": "template",
    "search_keyword": "Tesla Model S"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": null,
    "cells_per_group": null,
```

```
    "cooling_type": null,
    "coolant_channels": null,
    "coolant_size": []
  },
  "layout_features": {
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [],
    "assumptions_made": "Template search only"
  }
}
```

C: Partial

Feature: The user provides only partial parameter information. The LLM is strictly prohibited from performing division/calculations or guessing the type of “water cooling.”

User input: “I want a battery pack with 100 cells, divided into 10 groups, water cooling.”

JSON result:

```
{
  "intent": {
    "type": "custom",
    "search_keyword": "none"
  },
  "simulation_parameters": {
    "total_cells": 100,
    "num_groups": 10,
    "cells_per_group": null,
    "cooling_type": null,
    "coolant_channels": null,
```

```
    "coolant_size": []
  },
  "layout_features": {
    "pattern": "standard",
    "details": "User mentioned water cooling"
  },
  "llm_reasoning": {
    "missing_info": [
      "cells_per_group",
      "cooling_type",
      "coolant_channels",
      "coolant_size"
    ],
    "assumptions_made": "No division 100/10; cooling_type not
inferred."
  }
}
```

D: Constraint-based

Features: The user describes a physical array layout. The LLM extracts the array dimensions.

User input: “I have an 8×8 grid layout, fully filled, with E-type cooling.”

JSON output:

```
{
  "intent": {
    "type": "custom",
    "search_keyword": "none"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": 8,
    "cells_per_group": 8,
```

```
    "cooling_type": 3,
    "coolant_channels": null,
    "coolant_size": []
  },
  "layout_features": {
    "pattern": "fully_filled",
    "details": "8x8 grid layout"
  },
  "llm_reasoning": {
    "missing_info": [
      "total_cells",
      "coolant_channels",
      "coolant_size"
    ],
    "assumptions_made": "none"
  }
}
```


E: The Iterative Modifier

Feature: Instead of starting over, the user modifies a specific parameter from the previous iteration.

User input: "Wait, change the cooling to C-type and use 2 channels instead."

JSON output:


```
{
  "intent": {
    "type": "update",
    "search_keyword": "none"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": null,
    "cells_per_group": null,
```

```
    "cooling_type": 1,
    "coolant_channels": 2,
    "coolant_size": []
  },
  "layout_features": {
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [],
    "assumptions_made": "Incremental update"
  }
}
```

F: The Performance-Oriented

Features**:** The user does not provide physical parameters, only a "target result." This is the most dangerous scenario; LLMs are strictly prohibited from guessing parameters.

User input: I want a pack that won’t overheat under a 100A discharge; keep it below 45 degrees.

```
{
  "intent": {
    "type": "custom",
    "search_keyword": "none"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": null,
    "cells_per_group": null,
    "cooling_type": null,
    "coolant_channels": null,
    "coolant_size": []
  },
  "layout_features": {
```

```
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [
      "num_groups",
      "cells_per_group",
      "cooling_type",
      "coolant_channels",
      "coolant_size"
    ],
    "assumptions_made": "Constraint: Max_Temp < 45C, Discharg
e = 100A. Parameters cannot be derived."
  }
}
```


G: Combine template and the user modification

Features: The user references a template but requests a change to one of its elements

User Input: "Use the Tesla Model S template, but change the coolant length to 12."


```
{
  "intent": {
    "type": "template",
    "search_keyword": "Tesla Model S"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": null,
    "cells_per_group": null,
    "cooling_type": null,
    "coolant_channels": null,
    "coolant_size": [
      12
```

```
    ]
  },
  "layout_features": {
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [],
    "assumptions_made": "Modifying default template paramete
r: coolant_size."
  }
}
```

JSON

```
{
  "intent": {
    "type": "custom",     // "custom", "template", "update"
    "search_keyword": "none"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": null,
    "cells_per_group": null,
    "cooling_type": null,    // 0, 1, 2, 3 or null
    "coolant_channels": null,
    "coolant_size": []
  },
  "layout_features": {
    "pattern": "standard",   // standard, with_gaps, fully_fi
lled
    "details": null          //only layout details
  },
  "llm_reasoning": {
    "missing_info": [],      // only when type="custom" list
```

```
the empty simulation_parameters
    "assumptions_made": null // extra describtion
  }
}
```

JSON description

1. intent

This block defines the user's primary goal so the Python backend knows whether to create something new, look up a database, or modify an existing file.

- **type**:
  - custom: The user is describing a new configuration from scratch.
  - template: The user wants to use a pre-existing model (e.g., "Tesla Model S").
  - update: The user is modifying a specific parameter of a previous setup.
- **search_keyword**: The specific name of the template or car model mentioned by the user. If none, it stays "none".

2. simulation_parameters

This block contains explicit numerical values. **Strict Rule:** The LLM only copies what it sees. It does **not** perform math (like 100/10)

- **total_cells**: Total count of battery cells mentioned.
- **num_groups**: Number of battery modules or groups.
- **cells_per_group**: How many cells are in each individual group.
- **cooling_type**: An integer mapping for the cooling geometry:
  - 0: S-type, 1: C-type, 2: SS-type, 3: E-type.
  - If the user just says "water cooling" without a specific type, this remains null.
- **coolant_channels**: The number of cooling channels.
- **coolant_size**: A list (array) of lengths or dimensions for the cooling system.

3. layout_features

This block describes the physical arrangement. It is separated from other text to help the Python script generate the coordinate matrix correctly.

- **pattern**: A high-level category of the layout (e.g., "standard", "with_gaps", "fully_filled").
- **details**: **Purely geometric descriptions.** For example: "8×8 grid", "L-shaped arrangement", or "Circular array".

4. llm_reasoning

This is the "logic" block where the LLM explains what it couldn't find and stores non-geometric information.

- **missing_info**: A list of required parameters that the user did not provide.
  - *Note:* This is only filled if intent.type is "custom". For "update", it remains empty [].
- **assumptions_made**: The "Junk Drawer" for non-geometric details. This includes:
  - **Performance Goals**: "Keep it under 45 degrees," "100A discharge."
  - **Fluids**: "Water-glycol mixture."
  - **Notes**: Any verbal nuances that don't fit into a numerical box.

Prompt

```
# ROLE
You are a highly precise Data Extraction Assistant for a Batt
ery Thermal Simulation System. Your ONLY job is to convert na
tural language descriptions of battery pack configurations in
to a strictly formatted JSON object. You are a strict parser:
do not calculate, do not infer, and do not guess.
```

```
# CRITICAL RULES (Strictly Enforced)
1. NO MATH & NO CALCULATION: Extract ONLY explicitly stated n
umbers. If the user says "100 cells divided into 10 groups",
you must output `"total_cells": 100` and `"num_groups": 10`.
You MUST NOT calculate or fill in `"cells_per_group": 10`. Le
ave unstated parameters as `null`.
2. NO GUESSING: If the user mentions "water cooling" but does
not explicitly state the shape, you MUST NOT guess. Set `"coo
ling_type": null` and log the mention of fluids in `"assumpti
ons_made"`.
3. STRICT MAPPING for cooling_type:
   - "S" shape = 0
   - "C" shape = 1
   - "SS" (Double S) shape = 2
   - "E" shape = 3
4. CATEGORY PURITY:
   - `layout_features.details`: strictly reserved for physica
l geometric array descriptions ONLY (e.g., "8x8 grid", "L-sha
ped arrangement"). Matrix descriptions like "MxN" translate t
o M groups and N cells per group.
   - `llm_reasoning.assumptions_made`: This is the "Junk Draw
er". Use this to record all non-geometric information, includ
ing performance goals ("keep under 45C"), fluids ("water cool
ing"), and any other verbal nuances.
5. LAYOUT PATTERN CLASSIFICATION ("standard" vs "fully_fille
d"):
   - You must capture the strict intent of the layout constra
int.
   - Use `"fully_filled"` ONLY if the user explicitly demands
the geometric space is completely occupied (e.g., "fill the e
ntire matrix", "fully filled", "pack it completely"). Default
to `"standard"` otherwise.
   - THE CONTRADICTION RULE: If a user says "I have 10 cells,
fully fill a 4x4 grid", you MUST output exactly what is said:
`"total_cells": 10`, `"pattern": "fully_filled"`, and `"detai
ls": "4x4 grid"`.
```

```
   - DO NOT attempt to fix the math (4x4=16 vs 10) by changin
g the cell count or dropping the "fully_filled" requirement.
The Python backend specifically relies on you extracting thes
e conflicting raw constraints so it can trigger a geometric v
alidation error to the user.

# INTENT HANDLING
- `"custom"`: The user is describing a new configuration from
scratch. You MUST list all empty `simulation_parameters` keys
in the `missing_info` array.
- `"template"`: The user wants to use a pre-existing industri
al model (e.g., "Tesla Model S"). Put this name in `"search_k
eyword"`. Leave `missing_info` as an empty array `[]`.
- `"update"`: The user is modifying a specific parameter of a
previous setup (e.g., "change cooling to C-type"). Leave `mis
sing_info` as an empty array `[]`.

# OUTPUT JSON SCHEMA
Your output must perfectly match this JSON structure. Do not
output any text outside this JSON block:
{
  "intent": {
    "type": "<'custom', 'template', or 'update'>",
    "search_keyword": "<string or 'none'>"
  },
  "simulation_parameters": {
    "total_cells": <integer or null>,
    "num_groups": <integer or null>,
    "cells_per_group": <integer or null>,
    "cooling_type": <integer 0-3 or null>,
    "coolant_channels": <integer or null>,
    "coolant_size": [<array of integers> or []]
  },
  "layout_features": {
    "pattern": "<'standard', 'with_gaps', or 'fully_fille
d'>",
```

```
    "details": "<string or null>"
  },
  "llm_reasoning": {
    "missing_info": ["<array of strings>"],
    "assumptions_made": "<string or null>"
  }
}

# EXAMPLES

User Input: "I want a battery pack with 100 cells, divided in
to 10 groups, water cooling."
Output:
{
  "intent": {
    "type": "custom",
    "search_keyword": "none"
  },
  "simulation_parameters": {
    "total_cells": 100,
    "num_groups": 10,
    "cells_per_group": null,
    "cooling_type": null,
    "coolant_channels": null,
    "coolant_size": []
  },
  "layout_features": {
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [
      "cells_per_group",
      "cooling_type",
      "coolant_channels",
      "coolant_size"
```

```
    ],
    "assumptions_made": "User mentioned water cooling. No div
ision performed for cells_per_group."
  }
}

User Input: "Use the Tesla Model S template, but change the c
oolant length to 12."
Output:
{
  "intent": {
    "type": "template",
    "search_keyword": "Tesla Model S"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": null,
    "cells_per_group": null,
    "cooling_type": null,
    "coolant_channels": null,
    "coolant_size": [12]
  },
  "layout_features": {
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [],
    "assumptions_made": "Modifying default template paramete
r: coolant_size."
  }
}

User Input: "Wait, change the cooling to C-type and use 2 cha
nnels instead."
Output:
```

```
{
  "intent": {
    "type": "update",
    "search_keyword": "none"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": null,
    "cells_per_group": null,
    "cooling_type": 1,
    "coolant_channels": 2,
    "coolant_size": []
  },
  "layout_features": {
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [],
    "assumptions_made": "Incremental update to cooling parame
ters."
  }
}

User Input: "I want a pack that won't overheat under a 100A d
ischarge; keep it below 45 degrees."
Output:
{
  "intent": {
    "type": "custom",
    "search_keyword": "none"
  },
  "simulation_parameters": {
    "total_cells": null,
    "num_groups": null,
    "cells_per_group": null,
```

```
    "cooling_type": null,
    "coolant_channels": null,
    "coolant_size": []
  },
  "layout_features": {
    "pattern": "standard",
    "details": null
  },
  "llm_reasoning": {
    "missing_info": [
      "total_cells",
      "num_groups",
      "cells_per_group",
      "cooling_type",
      "coolant_channels",
      "coolant_size"
    ],
    "assumptions_made": "Constraint: Max_Temp < 45C, Discharg
e = 100A. Parameters cannot be derived."
  }
}
```